IMPLEMENTATION PROMPT / 01

Build a focused grant workspace

Paste the companion TXT file into Codex. Everything from the next paragraph through the delivery checklist is the implementation brief. Build the application; do not stop at a plan.

Your task

Implement GrantPath, a working single-user research-grant preparation app. Use https://github.com/oliverchennn/hackathon-kit for source reuse, conventions and delivery discipline. The product should help a researcher turn project evidence and a funding call into a source-checkable, editable application plan and draft. This is a deliberately small first iteration. Make reasonable implementation decisions and continue through integration and verification.

Exactly four connected capabilities

1. Project profile and a small text evidence library.
2. Explainable matching against six synthetic example opportunities and user-added calls.
3. An application checklist with document statuses, deadlines, an agenda and calendar-file export.
4. Three editable proposal sections, optional live AI drafting, and deterministic application-readiness checks.

Build one complete journey across these four capabilities. Do not add separate unfinished modules just to make the navigation look comprehensive.

The user journey

Create a project -> add evidence -> inspect opportunity matches -> save one opportunity as an application -> confirm requirements and deadlines -> draft and edit sections -> resolve readiness issues -> export the draft and checklist. The same project facts, source versions and application records must flow through every step.

Product experience

Use a calm, readable research-workspace interface with a project header, compact navigation, and an application detail view. Display evidence alongside the item it supports. Provide clear empty, saving, saved, loading and recoverable-error states. Use accessible labels, keyboard controls, visible focus and sensible mobile stacking. Make important actions real: save, edit, confirm, regenerate, export and open source. No decorative charts or fake success messages.

Hard scope boundary

Do not build live grant crawling, institutional authentication, multi-user collaboration, scientific-merit grading, award-probability scores, resubmission automation, ethics approval processing, external reminders, budget accounting, post-award reporting, Bootstrap, GrantWeave or AimFit. Do not submit applications, send messages, deploy or publish. Implement local execution only. Do not install or operate the kit's cross-computer orchestration system.

IMPLEMENTATION PROMPT / 02

Use the kit deliberately

Inspect the actual repository and selected modules before implementing against their interfaces. The inspected planning reference is commit 22a5f44a27375e88ac2cd1754f6976704f725bc5.

Inspect and preserve

Inspect the target directory, Git state, framework, package manager and existing user work. Read the kit's AGENTS.md, docs/start-here/AGENT_START.md and relevant scoped instructions. Read docs/start-here/CREATE.md or ADOPT.md, CATALOG.md, and the README, module.json and minimalRead files for documents, extraction, recommendations and ai. Also use the task-card, ownership and API-contract templates. Record the actual kit revision used. If APIs differ from this brief, adapt to the checked-out version and document the difference.

Create only a lean dependency closure

For a new app, use the kit CLI from an authorized checkout and an empty destination outside the kit. Use Node 22.23+ or the checked-out kit's declared minimum. A starting command sequence is:
npm ci
npm run kit -- inspect documents
npm run kit -- inspect extraction
npm run kit -- inspect recommendations
npm run kit -- inspect ai
npm run kit -- create <absolute-empty-app-dir> --starter web --modules documents,extraction,recommendations,ai
Choose the destination yourself based on the workspace; the angle-bracket path is a placeholder, not a literal command argument. Follow the generated README for installation, checks and startup. Do not use the full recipe.

If a target app already exists

Use the kit's plan/add workflow and review conflicts before the explicit --apply step. Preserve existing routes, storage, package-manager conventions and agent instructions. For an unsupported framework, integrate only portable selected source with documented changes. Do not overwrite the source toolkit or create symlinks back to it. Preserve LICENSE, THIRD_PARTY_NOTICES and generated provenance/hashes.

Reuse strengths; handle the actual limits

documents supplies normalized text and source positions. extraction supplies labeled-line parsing, ambiguity states and evidence validation; it is not a general grant-reading engine. recommendations supplies deterministic ranking primitives; its default missing-hard-metric behavior must not turn unknown eligibility into a confirmed mismatch. ai supplies server-only providers, structured output and explicit fixture mode. Use these modules rather than parallel homemade infrastructure, while adding the grant-domain adapters they lack.

Keep the setup small

Do not add embeddings, vector storage, OCR, Python, background workers, Supabase, real-time sync or external integrations in this iteration. Do not run npm run agents or adopt docs/guides/ORCHESTRATION.md. Keep only concise project-specific planning records; the kit templates are aids, not a requirement to generate an elaborate process system. If repository access is genuinely unavailable, state that blocker; do not pretend the kit was used.

IMPLEMENTATION PROMPT / 03

Profile, evidence and matching

Make user-provided facts and opportunity requirements inspectable. Unknown information must remain unknown.

Capability 1: project and evidence workspace

Capture project title, short abstract, topic tags, institution/applicant type, career stage, country/region, approximate funding need with currency, intended dates and known eligibility facts. Permit unknown values. Save source notes for preliminary work, methods, team qualifications and facilities. Require pasted text or UTF-8 TXT/Markdown intake only; defer PDF and DOCX upload to the roadmap.

Evidence behavior

Each source has a stable ID, title, type, text, version/hash, added/updated time, optional source URL, and provenance such as researcher-provided or demo. Show the exact normalized text used for extraction and drafting. Snapshot source, project and confirmed-call revisions used by drafts/checks. Retain referenced versions; material edits mark dependent output stale and recompute matches. Never interpret an upload, a user claim or a citation as independent verification. Start with at most ten sources per project and 20,000 characters per source; show actionable limit errors.

Capability 2: small explainable grant shortlist

Seed exactly six clearly fictional demo calls with synthetic source text. Include plausible fit, applicant-type mismatch, career-stage mismatch, unknown eligibility, an expired call, and an ambiguous deadline. Label demo data at record and workspace level; never imply these grants exist. Let users add a real call by pasting its text and entering title, funder and optional official URL. Saving a URL does not fetch or verify it. Preserve importedAt separately from a user-confirmed lastCheckedAt.

Matching rules

Evaluate only explicit, confirmed criteria. Known violations yield Known mismatch; unresolved facts yield Needs confirmation. Potential fit requires a reviewed eligibility assessment and matches on all modeled required criteria; other call conditions may still apply. Unreviewed calls or empty extractions default to Needs confirmation, never a vacuous pass. Closed/expired calls are not shown as open. Rank potential fits using transparent topic and funding-range preferences. Keep unknown calls visible separately. Never turn unknown data into a pass, invent a rule or show a funding probability.

What the researcher sees

For every result show the status, matched project facts, relevant call quotes, unresolved criteria, amount/currency, deadline status and source freshness. Users can inspect or correct structured call fields and save an opportunity to an application. Duplicate saves reopen the existing application for that project/call. If there are no suitable calls, show the reasons and allow adding a call; do not silently relax hard criteria.

Done when

A profile edit updates explanations; the synthetic mismatch never appears as an unqualified fit; an unknown institution remains unresolved; the expired example is distinguishable; and a user-added call can enter the same workflow as a seed record. All edits survive refresh and server restart.

IMPLEMENTATION PROMPT / 04

Requirements and deadlines

A researcher should be able to see what is required, what is missing, and which date actually matters.

Capability 3: one application workspace

Use stages Considering, Preparing, Ready for review and Submitted. Submitted is a user-set record only; no external submission takes place. Saving a call creates a workspace with requirements, supporting-document statuses, internal tasks and sponsor deadline. Track a plain-text responsible person if useful, without creating accounts or assigning external notifications.

Extract, review, confirm

Use labeled-line extraction for supported structured call text. For prose, provide a simple manual action to create a requirement from selected or pasted exact source text. With live AI configured, optionally suggest structured requirements, but validate each quote against the saved source and require researcher confirmation. Never present an empty extraction as proof that no requirements exist. Mark incomplete call review explicitly.

Requirement data and behavior

Store category, description, required/optional/unknown status, source ID/version and quote/span, review status, task status, due-date metadata, optional word limit, and linked evidence/document record. Distinguish sponsor-required items from user-added tasks. Proposed extraction is not a confirmed requirement. Show conflicting or absent values as unresolved. Manual unsupported overrides remain visibly user-entered. Editing the call marks prior extraction stale.

Documents and ethics in this version

Track document items as Missing, In progress or Ready, with a title, optional reference/link and note. Users may link evidence notes already in the app. This is metadata tracking, not a file vault. An ethics-related requirement can be listed, but the app cannot approve a study. Unknown approval stays unknown; Ready records the user's document status and never means that the app certified compliance.

Dates that do not guess

Preserve original deadline wording. Represent deadline as unknown, date-only, or exact timestamp with confirmed source timezone/offset. Require confirmation for ambiguous dates and missing timezones. Keep the official deadline separate from internal targets. Show an agenda with upcoming, overdue and unresolved-date items. Date-only items must say that the cutoff time is unknown; do not invent midnight or 11:59 PM.

Calendar and readiness behavior

Export one confirmed event as .ics with a stable UID and correct escaping. A date-only event is all-day with an exclusive next-day DTEND and a note that the cutoff time is unknown. Exact timestamps export in UTC after validated conversion. Block timed export when timezone is unresolved. Explain that file import is not continuous calendar sync. In-app reminders are badges/agenda entries only; no email, push, background scheduler or promised closed-app reminder.

IMPLEMENTATION PROMPT / 05

Drafts and readiness checks

Help researchers prepare and inspect their work. Do not claim to determine scientific merit or funding outcomes.

Capability 4: three editable sections

Provide Project summary, Significance and Approach overview, with editable labels/word limits when the call uses different names. These are a starter draft, not a complete arbitrary sponsor application. Manual editing works without AI. A researcher can choose evidence sources, generate one section, inspect the proposed text and support, and accept it. Preserve the previous version; regeneration must not overwrite unsaved or accepted edits silently.

Grounded generation contract

Pass the selected project facts, confirmed call requirements and selected source text to the server-only ai adapter. Request a validated object containing section text, references to source IDs/versions and exact quotes, and missingInputs. Validate source identity and quote existence before rendering accepted output. Keep source references next to the draft and in a separate export appendix. A valid quote link is provenance, not proof that the scientific conclusion is correct; label the output Researcher review required.

Missing evidence and untrusted text

Never invent preliminary results, sample sizes, publications, approvals, collaborators, completed experiments or funding commitments. Missing facts become explicit input requests or clearly marked placeholders. Treat calls and documents as data, not instructions that can change the system prompt, invoke tools or reveal secrets. No web research or autonomous external tool calls in drafting. Reject fabricated source references and surface a recoverable generation error.

Deterministic readiness review

Check confirmed required sections/items, empty draft sections, unresolved placeholders, declared word limits, missing supporting-document statuses, unresolved eligibility/deadlines, unreviewed requirements, invalid source links and stale source versions. Return issues with severity, location and a fix action. Use blockers/warnings and counts, not a global grade. Only mark Ready for review after known blockers are addressed or explicitly waived with a reason. Manual status is not evidence that an application is complete under every sponsor rule.

Live AI and no-key operation

Keep provider/model/keys on the server. Read actual environment names from the kit; support AI_PROVIDER, AI_MODEL and the selected provider key. Use explicit live opt-in, a user-triggered generation action, bounded output, timeout/cancellation and rate limits. Bound the full request to 50,000 characters; ask the user to reduce selected evidence if exceeded rather than silently truncating it. Avoid request/response logging of private content. Missing keys leave the manual app fully usable. A separate synthetic fixture demo is allowed only with visible Demo/fixture labels and isolated demo data; never substitute a fixture after a live failure.

Export and errors

Download a UTF-8 Markdown packet containing the current draft, application checklist, unresolved issues and source appendix. Identify placeholders and unresolved requirements. Preserve edits on timeouts, invalid JSON, rate limits and cancellations; offer retry. Default tests must use deterministic fixtures or mocked transport, never paid provider calls.

IMPLEMENTATION PROMPT / 06

Contracts and local persistence

Prefer a small coherent data model over a general grants-management framework. Establish shared contracts before parallel feature work.

Core records

Project: profile, research facts, topic tags, funding need/currency and timestamps.
SourceDocument/SourceVersion: canonical text, source metadata, normalized-text hash, version and provenance.
Opportunity: call source version, structured fields, explicit criteria, confirmation/freshness state and deadline.
Application: project/opportunity IDs, stage and selected source versions.
Requirement/Task: source support, confirmation, status, document reference, due date and optional word limit.
DraftSection/Revision: section type, text, source references, project/call input revisions, missing inputs, generation mode, prompt version and timestamps.
ReadinessResult: rules version, application revision, issue list and evaluation time. Prefer derived checks to duplicated mutable truth.

Shared evidence and deadline types

A citation identifies source ID, source version/hash, exact quote and normalized-text offsets; preserve the kit's UTF-16, end-exclusive convention. Store an optional section/page label only when known. A deadline discriminated union must prevent an unconfirmed date from masquerading as a precise timestamp. Use ISO dates, explicit currency codes, stable IDs and Zod schemas at boundaries. Invalid identifiers, stale versions and malformed provider output return actionable errors.

Persistence choice

Reuse an existing reliable store if present. For a new local single-user app, use one server-side JSON repository with serialized mutations, atomic temporary-file rename and schemaVersion; do not build an ORM or database service. Keep data in a gitignored local directory, never in the source-controlled seed file. Retain source/draft versions needed by current references. Detect revision conflicts instead of silently overwriting a newer edit. Bind the app to loopback; this storage/authentication choice is not a hosted multi-user design.

Boundaries and resources

Expose application-owned server routes with input validation, request-size limits and consistent errors. Keep provider adapters and file IO out of browser bundles. For local mutation routes validate same-origin requests and expected content types. Use safe text rendering, not raw HTML from calls or sources. Route all storage mutations through the shared repository. No file path from a user may become an unrestricted filesystem target.

Fixtures and reset

Create one coherent synthetic research project with source notes and six calls. The same fixtures should drive matching, requirement extraction and the draft demo. Separate user records from seeded demo records. A visible reset action clears demo data only and states its effect; it must not wipe a user-entered project. Persist normal user data across process restarts. Do not present fixture completion as live model verification.

IMPLEMENTATION PROMPT / 07

Use local agents efficiently

Use agents on this machine for independent implementation after contracts are stable. One lead integrates and verifies the result.

Lead setup before delegation

The lead inspects the workspace and kit, creates/adopts the app, chooses storage, establishes types/API contracts and shared fixtures, and gets one thin save/load route working. Fill concise PROJECT_BRIEF, MODULE_SELECTION, OWNERSHIP, API_CONTRACT, TASK_BOARD and DEMO_RUNBOOK records using the generated project conventions. Record implementation progress and actual verification. Treat the user as product owner; do not invent extra human teammates or pause to demand three people.

Bounded parallel lanes

Agent A owns profile/evidence UI and opportunity matching, with its domain adapter and focused tests.
Agent B owns the application checklist, document statuses, agenda, .ics serializer and focused tests.
Agent C owns server-side drafting, draft editor, deterministic readiness engine and focused tests.
The lead owns shared schemas, repository/storage, migrations if any, app shell/navigation, global styles, package manifests/lockfile, shared fixtures and integration. Give each agent concrete exclusive file globs before writing; split component/route ownership explicitly.

Isolation and coordination

Give concurrent writers separate local Git worktrees/checkouts and branches. These are all on the same machine. Assign separate ports and data directories; worktrees do not isolate those resources automatically. Keep one authoritative task/ownership board in the lead checkout. Workers propose shared-contract or dependency changes to the lead instead of editing shared files. If isolated writer support is unavailable, serialize conflicting work and use agents for read-only review; document the fallback rather than running concurrent writes in one shared file set.

Task contracts and handoff

Each bounded task states read-first files, exact ownership, inputs/outputs/errors, fixtures, intended user behavior, tests and non-goals. Worker output includes changed files/commit, actual checks, limitations and integration notes. Integrate in dependency order, inspect the diff, resolve conflicts, and rerun affected checks. A worker's success message is not verification. Use an agent for a read-only final review of missed requirements and scope creep once the core flow works.

Execution order and scope control

First prove persistence and the user journey with manual input. Then complete matching, requirements and drafts in parallel. Integrate real routes early, add bounded optional AI, and finish failure states and exports. If time is tight, keep all four capabilities at their specified minimum: six calls, pasted text, three sections, in-app agenda and one-event export. Cut visual polish before provenance, persistence, the manual no-key path or functional integration. Do not start roadmap work even if one worker finishes early.

Explicit omission

Do not set up cross-computer workers, remote orchestration, shared distributed queues, GitHub coordination services, cross-account sessions or leader-election infrastructure. Use Codex's local agent capabilities and ordinary local branches/worktrees only. Preserve the kit's useful file-ownership and integration discipline without adopting its optional distributed system.

IMPLEMENTATION PROMPT / 08

Prove it works, then hand it over

Complete the implementation and report evidence. Do not call it production-ready or live-verified based only on fixtures.

Required acceptance gates

1. Create a project, paste real-shaped source text and a call, save an application, confirm requirements, edit a draft and export a packet. Refresh and restart the server; the accepted state persists.
2. Known mismatches, unknown eligibility, expired calls and demo calls remain distinguishable. Every explanation traces to inputs; no funding probability appears.
3. Requirement quotes and draft references resolve to the exact source version. Editing a source marks dependent output stale and preserves the older snapshot.
4. Ambiguous dates require confirmation. Date-only and exact timestamp .ics fixtures import with the intended dates, stable UIDs and correctly escaped text.
5. Over-limit text, malformed provider output, forged citations, timeout, missing key and a malicious instruction inside source text produce safe, recoverable behavior.
6. Regeneration preserves manual edits and previous revisions. Missing pilot results and ethics approvals never become fabricated accomplishments.
7. Readiness flags missing required items, word-limit violations, unresolved placeholders and stale evidence. It never implies peer review or sponsor acceptance.
8. Demo reset does not delete user data. Duplicate saves and conflicting edits do not corrupt records.

Testing and verification

Run generated project checks, targeted selected-module tests, domain tests, production build and one browser end-to-end journey. Use the scripts actually provided by the generated project; do not blindly copy kit-maintenance commands into a downstream app. Add meaningful tests for eligibility tri-state behavior, evidence validation/staleness, persistence/revision conflicts, deadline conversion/export and readiness. Exercise the UI with keyboard navigation and a narrow viewport. Check both the no-key path and a labeled fixture-AI path. Live transport tests require configured credentials and explicit operator intent; otherwise report them unrun.

Deliverables in the target project

Provide the runnable application; concise README with exact setup/start/check commands; environment example without secrets; coherent demo fixtures; source revision and reuse notes; and a short demo runbook. Include actual test/build results and any skipped prerequisites. State what is implemented, what is locally verified and what is not live-verified. List remaining limits and the next smallest follow-up. No deployment, publication or application submission is authorized by this brief.

Definition of finished

The end-to-end manual workflow is usable without external credentials. Optional AI works through a real bounded adapter when configured, with explicit modes and recoverable errors. Each visible core action works. The app is coherent enough to prepare one application, with honest support for missing evidence and uncertain requirements. Stop here rather than adding the future roadmap.